

Neuro Matrix: A Lightweight Supervised Matrix (LSM) Model for Adaptive Prediction Based on Statistical Transformations

Abstract

Neuro Matrix is a lightweight supervised matrix-learning model designed to perform adaptive prediction using statistical transformations instead of conventional neural-network architectures. The method applies sigmoid normalization to input and target matrices, measures their dependence using Karl Pearson's correlation coefficient, and generates predictions through a correlation-guided transformation. Prediction quality is evaluated using Root Mean Square Error (RMS), while an adaptive parameter is iteratively updated until the error converges.

1. Mathematical Model

Let the input be a column matrix (vector)

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \quad y' = \begin{bmatrix} y'_1 \\ y'_2 \\ \vdots \\ y'_n \end{bmatrix} \in \mathbb{R}^n$$

and let y' denote the target output column matrix. Sigmoid normalization is applied element-wise to map all values into $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad X_s = \sigma(X), \quad y'_s = \sigma(y').$$

The statistical relationship between normalized input X_s and current output y is measured using Karl Pearson's correlation coefficient:

$$r = \frac{\text{Cov}(X_s, y)}{\sigma_{X_s} \sigma_y}.$$

The lambda function in Neuro Matrix is used to generate an intermediate representation that gradually shifts the normalized input towards the normalized target, with the amount of adjustment controlled by correlation strength and an adaptive learning parameter.

$$\lambda = X_s + kr(y'_s - X_s)$$

Here λ is the intermediate transformed column matrix obtained by adjusting the normalized input toward the normalized target output, and $k \in \mathbb{R}$ is the adaptive learning parameter that controls the intensity of this update. The predicted output column matrix is obtained as

$$y = \sigma(\lambda).$$

2. Error Minimization

Prediction accuracy is measured using the Root Mean Square Error:

$$\text{RMS} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - y'_{s,i})^2}.$$

After each prediction step, the adaptive parameter is updated as

$$k_{\text{new}} = k_{\text{old}} - \alpha \text{RMS},$$

where $\alpha > 0$ is a small learning-rate constant that controls the speed of adjustment. The process of computing y , evaluating RMS, and updating k is repeated until the error converges to a stable minimum.